

Codex Build Goal: IZ Clinical Notes Analyzer — Clean Windows 11 Home Rebuild

Target model: GPT-5.5 Extra High

Target platform: OpenAI Codex App on macOS

Output target: A fully clean, resilient, fault-tolerant Windows 11 Home local laptop application

Date authored: 2026-06-14

Source repo: `IZ_clinical-notes-analyzer-main.zip` (attached to this session)

INSTRUCTIONS FOR CODEX

Read this document completely before writing a single line of code. Inspect every file in the attached repo before making any structural decisions. When you are uncertain whether to keep or discard a file, keep it and note the ambiguity. Do not invent requirements that are not stated here or in the existing codebase. Do not guess at clinical rules — use exactly what is in `config/rules/` and `config/checklists/`. Work incrementally: build, run tests, confirm, then proceed. Do not skip validation steps.

1. PROJECT CONTEXT

This is a **local-first clinical compliance tracking app** for R3 Recovery Services. The primary user is Marleigh (Clinical Director) who needs to track treatment plan timeliness for 60+ active behavioral health clients. The app runs entirely on a Windows 11 Home laptop with no internet dependency during operation (except for optional Alleve EMR API connectivity that is already gated behind a readiness harness).

Core architecture (keep exactly as-is):

- **Backend:** Python/FastAPI with SQLite (default), JWT auth, RBAC, encrypted uploads, forensic audit logging
- **Frontend:** React 18 + TypeScript + Vite SPA, served by the FastAPI backend in desktop mode
- **Runtime:** `backend/app/desktop_main.py` — one `uvicorn` process at `http://localhost:8000`
- **Data directory:** `%LOCALAPPDATA%\IZ Clinical Notes Analyzer\` — all SQLite DB, uploads, logs, and .env live here, NOT in the repo
- **Rules engine:** Deterministic YAML rules in `config/rules/` — no LLM required for compliance decisions

What has already been built and validated (DO NOT break these):

- 84 backend pytest tests passing
 - 11 frontend Vitest tests passing
 - 42-step Marleigh-validated treatment plan checklist
 - Treatment Plan Timeliness tracker with PHP/IOP/OP/LOC rules
 - Patient note binder system with encrypted storage
 - Forensic audit log with hash chaining and CEF/FHIR payloads
 - JWT auth with RBAC (admin / manager / counselor)
 - Workflow state machine (Draft → Submitted → Reviewed → Approved)
 - Windows preflight, startup scripts, and release package builder
-

2. WHAT IS WRONG WITH THE CURRENT CODEBASE (WHY THIS REBUILD)

2A. Structural problems

Problem	File(s)	Impact
Monolithic frontend	<code>frontend/src/App.tsx</code> is 5,267 lines	Unmaintainable; Codex/IDE context exhausted on every edit
Monolithic API router	<code>backend/app/api/routes.py</code> is 2,616 lines	No domain isolation; any route change risks breaking unrelated routes
Dead/legacy scaffolding	<code>depricedated/</code> , <code>diag-build-tools/</code> , <code>walkthroughs/</code> , <code>video-extract/</code>	Clutters context, confuses AI tools
Dev-only deps in prod requirements	<code>requirements.txt</code> includes <code>psycopg[binary]</code> , <code>pyinstaller</code> , <code>pytest</code> , <code>httpx</code>	Wrong for Windows desktop users; installs unnecessary PostgreSQL driver
Multi-target startup scripts	macOS/Ubuntu/Docker/PostgreSQL scripts alongside Windows scripts	Non-technical Windows users see irrelevant scripts
No proper component hierarchy	All UI is one file; no <code>components/</code> , no <code>views/</code> , no type file	Cannot scale; prevents AI code review

2B. User experience problems

Problem	Impact on Marleigh
No proper status-based color coding in the work queue	Overdue items look the same as compliant items
Upload feedback lacks progress indication	User doesn't know if a 5MB PDF is uploading or hung
No persistent session state between browser refreshes	User must re-authenticate after F5
The app opens to a "Dashboard" but Marleigh's first job is the work queue	Wrong first screen for the primary user
Windows 11 native feel is missing	Segoe UI not used; no Fluent-inspired density or spacing
Error messages are generic	"Internal server error" gives no guidance to a non-technical user
No keyboard shortcuts or accessibility	Required for professional healthcare tooling

2C. Resilience problems

Problem	Risk
No SQLite WAL mode configuration at startup	Concurrent reads during heavy audit log writes can fail
No retry logic on API calls from the frontend	A single 503 during startup kills the session
No graceful shutdown of uvicorn on Windows	Leaving orphan processes prevents clean restart
Database migration is manual SQL file	No migration runner; schema drift is undetected

3. WHAT TO KEEP (EXACT FILES — DO NOT REMOVE)

Copy every file in this list to the clean project verbatim. These are validated, working implementations.

Backend core (keep all, no changes unless noted)

backend/app/__init__.py	
backend/app/main.py	# FastAPI app factory – keep exactly
backend/app/desktop_main.py	# Desktop runtime – keep exactly
backend/app/api/__init__.py	
backend/app/api/deps.py	# JWT auth deps – keep exactly
backend/app/api/api_config_routes.py	# Alleva API harness – keep exactly
backend/app/api/api_config_ui_routes.py	# Desktop HTML for API config – keep exactly
backend/app/api/clinical_notes_ui_routes.py	# Desktop intake HTML – keep exactly
backend/app/api/rules_routes.py	# YAML rules API – keep exactly
backend/app/core/__init__.py	
backend/app/core/audit_template.py	# 42-step checklist template – keep exactly
backend/app/core/config.py	# Settings/pydantic config – keep exactly
backend/app/core/security.py	# JWT/bcrypt – keep exactly
backend/app/db/__init__.py	
backend/app/db/base.py	
backend/app/db/bootstrap.py	# Schema migration compatibility – keep exactly
backend/app/db/session.py	
backend/app/models/models.py	# All SQLAlchemy models – keep exactly
backend/app/schemas/schemas.py	# All Pydantic schemas – keep exactly
backend/app/services/access_intel.py	
backend/app/services/api_connectivity.py	# Alleva API harness – keep exactly
backend/app/services/app_settings.py	
backend/app/services/audit.py	# Forensic audit with hash chaining – keep exactly
backend/app/services/emr_fhir.py	
backend/app/services/evaluation.py	
backend/app/services/llm_assist.py	

```
backend/app/services/patient_notes.py      # File upload/encryption – keep
exactly
backend/app/services/review_source_discovery.py
backend/app/services/rules_engine.py      # Deterministic YAML engine –
keep exactly
backend/app/services/runtime_checks.py
backend/app/services/secure_storage.py    # Fernet encryption – keep
exactly
backend/app/services/timeliness.py        # Treatment plan timeliness –
keep exactly
backend/app/services/timezone.py
backend/app/services/treatment_plan_checklist.py
backend/app/services/version.py
backend/app/services/workflow_definitions.py
backend/migrations/001_initial.sql
```

Backend tests (keep all)

```
backend/tests/conftest.py
backend/tests/test_api_connectivity.py
backend/tests/test_auth_flow.py
backend/tests/test_chart_audit_flow.py
backend/tests/test_config.py
backend/tests/test_forensic_audit_logging.py
backend/tests/test_patient_note_uploads.py
backend/tests/test_rules_engine.py
backend/tests/test_schema_bootstrap.py
backend/tests/test_security_hardening.py
backend/tests/test_smoke_script.py
backend/tests/test_system_and_emr_readiness.py
backend/tests/test_treatment_plan_checklist.py
backend/tests/test_treatment_plan_timeliness.py
backend/tests/test_user_management.py
backend/tests/test_workflow_definitions.py
```

Config/rules (keep all, verbatim)

```
config/rules/alleva_treatment_plan_completeness_rules.yaml
config/checklists/treatment-plan-v1.json
```

Windows scripts (keep all)

```
scripts/Start-IZ-Clinical-Notes-Analyzer.cmd
scripts/startup-windows-local.ps1
scripts/preflight-windows.ps1
scripts/setup-windows.ps1
scripts/build-windows-installer.ps1
scripts/start-windows-local.ps1
scripts/update-local-admin.ps1
scripts/test-alleva-api-connectivity.ps1
scripts/test-api-configuration-local.ps1
scripts/test-local-app-stack.ps1
```

Docs (keep selected)

```
docs/Windows-Deployment-and-Test-Guide-Version-1.md
docs/Windows-User-Guide-Version-1.md
docs/admin-access-reset.md
docs/architecture.md
docs/open-blockers.md
docs/runbook.md
docs/sample-clinical-notes/           # all files - synthetic test data
docs/alleva-clinical-note-example.md
docs/prd/prd_2026-06-11_updated-treatment-plan-comprehensive-prd.md
```

Metadata

```
VERSION
VERSION.json
pytest.ini
.env.example
.gitignore
AGENTS.md                          # update with clean-project
instructions
```

Example treatment plans (synthetic PDFs for testing)

```
example-treatment-plans/JTX.Pdf
example-treatment-plans/XTXP.pdf
```

4. WHAT TO EXCLUDE (DO NOT COPY TO CLEAN PROJECT)

These files are legacy, dev-only, or Docker-era scaffolding. A non-technical Windows 11 Home user should never see them.

```
depriced/ # entire folder – Docker/nginx era,
deprecated 2026-06-11
diag-build-tools/ # diagnostic sync scripts, not part of
app
video-extract (2026-06-05)/ # reference-only; already analyzed and
implemented
walkthroughs (2026-03-04)/ # screenshots; not runtime artifacts
.github/ # CI workflow; not needed for local
Windows build
docker-compose.db-expose.yml # Docker developer tool
docs/prd/prd_2026-06-01_ver0.0-old-original.rtf # legacy, superseded
docs/prd/prd_2026-06-01_treatment-plan-timeliness-mvp.md # superseded by
June 11 PRD
docs/chart-review-workflow-codex-build-prompt.md # historical build
prompt
docs/CODEX_COMPLETION_LOG.md # internal dev log, not operator doc
docs/cleanup-audit.md # dev audit log
docs/codebase-map.md # dev map, will be replaced by new
structure
docs/codex-build-readiness-log.md # dev log
docs/codex-goal-treatment-plan-mvp-2026-06-01-0800.md # old goal
docs/codex-goal-treatment-plan-ui-api-completion-2026-06-11.md # old goal
docs/removal-log.md # dev log
docs/version-1-completion-log.md # dev log
docs/version-1-completion-plan.md # dev plan
docs/version-1-final-validation-report.md # superseded
docs/validation/ # internal validation logs
docs/windows-local-refactor.md # dev refactor notes
docs/windows-startup-known-issue-20260514.md # resolved issue
scripts/startup-macos.sh # macOS only – not Windows 11
scripts/startup-ubuntu-24.04.sh # Linux only – not Windows 11
scripts/startup-windows.ps1 # older Docker/PostgreSQL Windows
path, superseded
scripts/start-desktop-local.ps1 # superseded by startup-windows-
local.ps1
scripts/smoke.sh # bash – not Windows
scripts/lib/dedicated-postgres.sh # PostgreSQL only – not needed for
SQLite desktop
"App Credentials Info.md" # security risk – replaced with
placeholder
```


5. NEW REQUIREMENTS FILE

Create `backend/requirements-windows.txt` (replaces both existing requirements files for the Windows desktop target):

```
# IZ Clinical Notes Analyzer – Windows 11 desktop requirements
# Python 3.11+ required. No Docker, PostgreSQL, or system packages needed.
fastapi==0.136.1
uvicorn==0.30.6
SQLAlchemy==2.0.35
python-jose[cryptography]==3.5.0
passlib[bcrypt]==1.7.4
bcrypt==4.0.1
python-multipart==0.0.27
pydantic-settings==2.5.2
email-validator==2.2.0
pypdf==6.10.2
cryptography==47.0.0
PyYAML==6.0.2
httpx==0.28.1
# --- dev/test (not installed in release package) ---
pytest==9.0.3
```

Justification for removals:

- `psycopg[binary]` — PostgreSQL driver; not needed for SQLite-first Windows desktop
- `pyinstaller` — packaging tool, not a runtime dependency; belongs in build scripts only
- `uvicorn[standard]` → `uvicorn` — the `[standard]` extras add `websockets` and `httptools` which are not required for this app

Keep `httpx` in the base requirements because it is used by `api_connectivity.py` and tests.

6. FRONTEND REFACTORING (CRITICAL)

The existing `frontend/src/App.tsx` (5,267 lines) **must** be decomposed into a proper component hierarchy. This is the single most important structural change. The existing functionality must be preserved exactly; this is a reorganization, not a rewrite.

6A. Target file structure

```
frontend/src/
├─ main.tsx                # keep exactly as-is (5 lines)
├─ app.css                 # keep exactly as-is – do not modify
CSS
├─ App.tsx                 # NEW: root component + view router
ONLY (~80 lines)
├─ types/
│   └─ index.ts            # NEW: all shared TypeScript types
extracted from App.tsx
├─ api/
│   └─ client.ts           # NEW: all fetch() wrappers extracted
from App.tsx
├─ components/
│   └─ layout/
│       └─ Sidebar.tsx     # NEW: sidebar navigation extracted
from App.tsx
│       └─ TopBar.tsx      # NEW: top bar extracted from App.tsx
│       └─ common/
│           └─ StatusBadge.tsx # NEW: timeliness/compliance status
badge
│           └─ LoadingSpinner.tsx # NEW: reusable loading state
│           └─ ErrorBanner.tsx    # NEW: error display with retry
│           └─ ConfirmDialog.tsx  # NEW: confirmation modal
│           └─ Toast.tsx          # NEW: toast notification (replaces
alert())
│       └─ timeliness/
│           └─ TimelinessQueue.tsx # NEW: work queue list panel
│           └─ ClientDetailPanel.tsx # NEW: right-side client evidence
detail
│       └─ EvidenceComparisonTable.tsx # NEW: document/signature/LOC date
comparison
├─ views/
│   └─ DashboardView.tsx    # EXTRACTED from App.tsx
│   └─ TimelinessView.tsx   # EXTRACTED from App.tsx – primary
Marleigh screen
│   └─ ReviewsView.tsx      # EXTRACTED from App.tsx
│   └─ UploadsView.tsx      # EXTRACTED from App.tsx
│   └─ ChecklistView.tsx    # EXTRACTED from App.tsx
│   └─ UsersView.tsx        # EXTRACTED from App.tsx
│   └─ LogsView.tsx         # EXTRACTED from App.tsx
│   └─ SettingsView.tsx     # EXTRACTED from App.tsx
```

	└ ProfileView.tsx	# EXTRACTED from App.tsx
└	test/	
	└ setup.ts	# keep exactly as-is

6B. Extraction rules

1. **Types:** Every `type` and `interface` declaration in `App.tsx` → `src/types/index.ts`. Export all.
2. **API calls:** Every `fetch()` call → `src/api/client.ts` as named async functions. The `const API` constant moves here.
3. **View components:** Each `AppView` case (dashboard, reviews, timeliness, checklist, uploads, profile, users, logs, settings) → its own file in `src/views/`.
4. **Layout components:** The sidebar and top bar markup → `src/components/layout/`.
5. **App.tsx becomes:** Import/render Sidebar, TopBar, and the active view based on `currentView` state. Handle auth state at this level. Nothing else.
6. **No logic changes:** The fetch URLs, API parameters, response handling, state shape, and render output must be identical to what existed in the original `App.tsx`. This is a structural reorganization only.
7. **No new dependencies:** Do not add React Router, Zustand, Redux, or any state management library. The existing `useState` pattern is correct for this app's scale.
8. **Preserve all existing tests:** `frontend/src/App.test.tsx` must still pass after the refactoring.

6C. Windows 11 UX improvements (implement in addition to extraction)

These are **additive UX improvements** that do not change backend behavior. Implement them during the refactoring.

1. Replace `window.alert()` and `window.confirm()` with proper UI components

The existing codebase uses browser `alert()` and `confirm()` for errors and confirmations. On Windows 11, these appear as ugly browser dialogs. Replace with:

- `<Toast>` component for transient success/error feedback (3-second auto-dismiss)
- `<ConfirmDialog>` component for destructive action confirmations
- `<ErrorBanner>` component for API errors that persist until dismissed

2. Upload progress feedback

Wrap file uploads in a progress-aware flow:

- Show a determinate progress bar using `XMLHttpRequest` with `upload.onprogress` (not `fetch`, which doesn't expose upload progress)

- Display file name, size, and (X of Y files) during multi-file uploads
- Show a spinner with "Analyzing..." state after upload while backend processes the files

3. Session persistence across refreshes

The existing app loses session on F5. Fix:

- Store the JWT token in (sessionStorage) (not (localStorage), per HIPAA guidance — tab-scoped only)
- On app load, check (sessionStorage) for an existing token before showing the login screen
- Call (GET /users/me) with the stored token to verify it's still valid
- If valid, restore session; if 401, clear and show login

4. Timeliness view as the default first screen for manager/admin role

- When a user with role (manager) or (admin) logs in, default (currentView) to ('timeliness') (not ('dashboard'))
- Counselors still default to ('uploads')

5. Status badge colors (enforce from (app.css))

Ensure every timeliness status badge uses consistent color coding from the visual style guide:

Overdue	→ coral/red	(#e66654 background, white text)
Urgent	→ orange	(#f59e0b background, white text)
Due Soon	→ amber	(#d97706 background, white text)
Returned	→ purple	(#7058f4 background, white text)
Needs Review	→ indigo	(#6366f1 background, white text)
Missing Data	→ gray	(#6b7280 background, white text)
Conflicting	→ dark gray	(#374151 background, white text)
Unable to Eval	→ slate	(#475569 background, white text)
Compliant	→ green	(#35c76f background, white text)
Approved	→ dark green	(#065f46 background, white text)

6. Windows 11 typography

Add to the top of (app.css):

```
css
```

```

:root {
  --font-sans: 'Segoe UI Variable', 'Segoe UI', system-ui, -apple-system, sans-serif;
  --font-mono: 'Cascadia Code', 'Consolas', 'Courier New', monospace;
}
body { font-family: var(--font-sans); }

```

7. Keyboard accessibility

- All interactive elements must be focusable with Tab
- Enter/Space must trigger button actions
- Escape must close modals and dialogs
- Add `aria-label` to icon-only buttons

7. BACKEND ROUTE REFACTORING

The `backend/app/api/routes.py` (2,616 lines) must be decomposed by domain. All existing route implementations must be preserved exactly — this is structural reorganization only.

7A. Target structure

```

backend/app/api/
├── __init__.py
├── deps.py # keep exactly as-is
├── api_config_routes.py # keep exactly as-is
├── api_config_ui_routes.py # keep exactly as-is
├── clinical_notes_ui_routes.py # keep exactly as-is
├── rules_routes.py # keep exactly as-is
├── routes/
│   ├── __init__.py
│   ├── auth.py # /auth/login, /auth/reset-password
│   ├── users.py # /users, /users/me, /users/{id}
│   ├── charts.py # /charts, /charts/{id}, /charts/{id}/transition,
etc.
│   ├── uploads.py # /note-sets, /note-sets/{id}/documents
│   ├── timeliness.py # /timeliness/dashboard, /timeliness/clients,
/timeliness/clients/{id}
│   ├── settings.py # /settings, /system/readiness, /emr/*, /ui-events
│   ├── audit.py # /audit-logs
│   └── workflow.py # /workflow-definitions, /workflow-
definitions/{id}/versions

```

7B. Registration in main.py

In `backend/app/main.py`, replace the single `router` import with the domain routers:

python

```
from app.api.routes.auth import router as auth_router
from app.api.routes.users import router as users_router
from app.api.routes.charts import router as charts_router
from app.api.routes.uploads import router as uploads_router
from app.api.routes.timeliness import router as timeliness_router
from app.api.routes.settings import router as settings_router
from app.api.routes.audit import router as audit_router
from app.api.routes.workflow import router as workflow_router

# Register all with prefix '/api'
for r in [auth_router, users_router, charts_router, uploads_router,
          timeliness_router, settings_router, audit_router, workflow_router]:
    api.include_router(r, prefix='/api')
```

7C. Shared helpers

All private functions in the current `routes.py` (those starting with `_`) that are used by multiple domains must move to `backend/app/api/shared.py`. Functions only used within one domain go into that domain's file.

8. RESILIENCE IMPROVEMENTS (BACKEND)

8A. SQLite WAL mode

Add to `backend/app/db/session.py` after engine creation:

python

```

from sqlalchemy import event
from sqlalchemy.engine import Engine

@event.listens_for(Engine, "connect")
def set_sqlite_pragma(dbapi_connection, connection_record):
    if 'sqlite' in str(engine.url):
        cursor = dbapi_connection.cursor()
        cursor.execute("PRAGMA journal_mode=WAL")
        cursor.execute("PRAGMA synchronous=NORMAL")
        cursor.execute("PRAGMA foreign_keys=ON")
        cursor.execute("PRAGMA busy_timeout=5000")
        cursor.close()

```

Justification: WAL mode allows concurrent readers during write operations. The existing default journal mode (DELETE) can return "database is locked" errors when audit logging writes overlap with dashboard reads. `busy_timeout=5000` gives 5 seconds before surfacing a lock error rather than failing immediately.

8B. Frontend API retry

In `src/api/client.ts`, wrap all GET requests with a simple retry on transient failures:

```

typescript

async function fetchWithRetry(url: string, options: RequestInit, retries = 2):
  for (let attempt = 0; attempt <= retries; attempt++) {
    try {
      const res = await fetch(url, options);
      if (res.status < 500 || attempt === retries) return res;
      await new Promise(r => setTimeout(r, 300 * (attempt + 1)));
    } catch (err) {
      if (attempt === retries) throw err;
      await new Promise(r => setTimeout(r, 300 * (attempt + 1)));
    }
  }
  throw new Error('Max retries exceeded');
}

```

Apply to GET calls only. POST/PATCH/DELETE must NOT be retried (not idempotent).

8C. Graceful Windows shutdown

Update `scripts/Start-IZ-Clinical-Notes-Analyzer.cmd` to register a Ctrl+C handler that stops uvicorn cleanly:

The existing `.cmd` file already handles this reasonably. Verify that `taskkill /PID` is used rather than `kill`, and that the process group is properly terminated so port 8000 is released before the next launch.

8D. User-facing error messages

In the FastAPI exception handlers in `main.py`, map status codes to non-technical messages for the Windows 11 user:

```
python

@api.exception_handler(500)
async def internal_error_handler(request, exc):
    logger.exception("Unhandled error on %s", request.url.path)
    return JSONResponse(status_code=500, content={
        "detail": "Something went wrong. Please close and reopen the app. "
        "If this keeps happening, contact your system administrator."
    })
```

9. CLEAN PROJECT FILE STRUCTURE

The final clean project must have exactly this structure (no extra files):

IZ-Clinical-Notes-Analyzer/	← root of new clean project
├─ AGENTS.md	← updated with clean project context
├─ VERSION	← 1.2.0
├─ VERSION.json	← updated
├─ .env.example	← keep as-is
├─ .gitignore	← keep as-is
├─ pytest.ini	← keep as-is
├─ README.md	← keep existing, update section on
file structure	
├─ CHANGELOG.md	← keep existing, add 1.2.0 entry
├─ backend/	
│ └─ requirements-windows.txt	← NEW clean requirements (see Section
5)	
│ └─ app/	
│ │ └─ __init__.py	
│ │ └─ main.py	← updated router registration
│ │ └─ desktop_main.py	← unchanged
│ │ └─ api/	
│ │ │ └─ __init__.py	
│ │ │ └─ deps.py	← unchanged
│ │ │ └─ api_config_routes.py	← unchanged
│ │ │ └─ api_config_ui_routes.py	← unchanged
│ │ │ └─ clinical_notes_ui_routes.py	← unchanged
│ │ │ └─ rules_routes.py	← unchanged
│ │ │ └─ shared.py	← NEW: shared helpers extracted from
routes.py	
│ │ │ └─ routes/	
│ │ │ │ └─ __init__.py	
│ │ │ │ └─ auth.py	
│ │ │ │ └─ users.py	
│ │ │ │ └─ charts.py	
│ │ │ │ └─ uploads.py	
│ │ │ │ └─ timeliness.py	
│ │ │ │ └─ settings.py	
│ │ │ │ └─ audit.py	
│ │ │ │ └─ workflow.py	
│ │ └─ core/	
│ │ │ └─ __init__.py	
│ │ │ └─ audit_template.py	← unchanged
│ │ │ └─ config.py	← unchanged
│ │ │ └─ security.py	← unchanged
│ └─ db/	
│ │ └─ __init__.py	

```

| | | └─ base.py                ← unchanged
| | | └─ bootstrap.py           ← unchanged
| | | └─ session.py             ← updated: WAL mode pragmas
| | └─ models/
| | | └─ __init__.py
| | | └─ models.py              ← unchanged
| | └─ schemas/
| | | └─ __init__.py
| | | └─ schemas.py             ← unchanged
| | └─ services/
| | | └─ [all 16 service files] ← all unchanged
| └─ migrations/
| | └─ 001_initial.sql          ← unchanged
| └─ tests/
| | └─ [all 16 test files]      ← all unchanged
└─ frontend/
    └─ index.html                ← unchanged
    └─ package.json              ← unchanged
    └─ tsconfig.json             ← unchanged
    └─ vite.config.ts            ← unchanged
    └─ src/
        └─ main.tsx              ← unchanged
        └─ app.css                ← updated: Windows 11 font stack added
        └─ App.tsx                ← NEW: root only (~80 lines)
        └─ types/
            └─ index.ts            ← NEW: all types from old App.tsx
        └─ api/
            └─ client.ts           ← NEW: all fetch() calls from old
App.tsx
    └─ components/
        └─ layout/
            └─ Sidebar.tsx
            └─ TopBar.tsx
        └─ common/
            └─ StatusBadge.tsx
            └─ LoadingSpinner.tsx
            └─ ErrorBanner.tsx
            └─ ConfirmDialog.tsx
            └─ Toast.tsx
        └─ timeliness/
            └─ TimelinessQueue.tsx
            └─ ClientDetailPanel.tsx
            └─ EvidenceComparisonTable.tsx
    └─ views/

```

```
|      |   ├── DashboardView.tsx
|      |   ├── TimelinessView.tsx
|      |   ├── ReviewsView.tsx
|      |   ├── UploadsView.tsx
|      |   ├── ChecklistView.tsx
|      |   ├── UsersView.tsx
|      |   ├── LogsView.tsx
|      |   ├── SettingsView.tsx
|      |   └── ProfileView.tsx
|      └── test/
|          └── setup.ts                ← unchanged
└── config/
    ├── rules/
    |   └── alleva_treatment_plan_completeness_rules.yaml ← unchanged
    └── checklists/
        └── treatment-plan-v1.json      ← unchanged
└── scripts/
    ├── Start-IZ-Clinical-Notes-Analyzer.cmd
    ├── startup-windows-local.ps1
    ├── preflight-windows.ps1
    ├── setup-windows.ps1
    ├── build-windows-installer.ps1
    ├── start-windows-local.ps1
    ├── update-local-admin.ps1
    ├── test-alleva-api-connectivity.ps1
    ├── test-api-configuration-local.ps1
    └── test-local-app-stack.ps1
└── docs/
    ├── Windows-Deployment-and-Test-Guide-Version-1.md
    ├── Windows-User-Guide-Version-1.md
    ├── admin-access-reset.md
    ├── architecture.md
    ├── open-blockers.md
    ├── runbook.md
    ├── alleva-clinical-note-example.md
    ├── sample-clinical-notes/
    |   └── prd/
    |       └── prd_2026-06-11_updated-treatment-plan-comprehensive-prd.md
└── example-treatment-plans/
    ├── JTXP.pdf
    └── XTXP.pdf
```

10. NON-NEGOTIABLE CONSTRAINTS (DO NOT VIOLATE)

These constraints are clinical, legal, and security requirements. Do not relax any of them.

10A. PHI and security rules

- Do not log, print, or expose uploaded note text, patient names, client identifiers, or document content anywhere in logs, audit records, browser responses, error messages, or test fixtures
- All uploaded files must remain encrypted at rest using the existing `secure_storage.py` Fernet envelope
- API keys, JWT secrets, bearer tokens, and encryption keys must never appear in browser responses, logs, console output, or test output
- Saved API credentials (LLM key, EMR secret) must be encrypted in the database using `encrypt_text_secret()` from `secure_storage.py`
- Do not use real PHI in any test fixture, synthetic sample, or screenshot

10B. LOC-change rule

- The treatment plan update window after a level-of-care change is **NOT CONFIRMED** by R3/Marleigh
- This setting must remain configurable via the admin Settings screen
- It must be visibly marked "Unvalidated — pending R3 confirmation" in the UI
- LOC-change cases must return `Needs Review`, `Missing Data`, or `Conflicting Evidence` — never silently `Compliant`
- Do not hard-code any number of days for LOC-change windows
- See `docs/open-blockers.md` for full context

10C. Live Alleva import gate

- Live patient import from Alleva remains **DISABLED**
- The API harness endpoints exist for connectivity testing only
- Do not add any code path that would write back to Alleva or import real patient records
- The existing readiness harness in `api_config_routes.py` is correct — do not modify it

10D. No Docker, no PostgreSQL, no external services

- The Windows 11 Home user must be able to run this app with Python only
- SQLite is the only required database

- Do not add any dependency that requires a background service, daemon, or network port other than 8000

10E. Deterministic compliance

- Every treatment plan compliance status must be deterministically derived from evidence fields
 - LLM assistance is optional and disabled by default; its setting is in the database admin settings
 - Do not add any AI decision path that can produce `Compliant` without documented evidence fields
-

11. UPDATED AGENTS.md (write this file)

After completing the build, write a new `AGENTS.md` that accurately describes the clean project. It must include:

1. **Repo purpose** — one paragraph summary of what this app does and who uses it
 2. **Architecture** — the clean file structure with one-sentence descriptions
 3. **How to run checks** — exact commands for backend tests, frontend tests, preflight, and launch
 4. **Security / PHI rules** — the non-negotiable list from Section 10A
 5. **LOC-change blocker** — copied from Section 10B
 6. **Alleva API boundary** — copied from Section 10C
 7. **Windows deployment** — how to launch, preflight, and build a release package
 8. **Do not add** — explicit list of things that must not be added (Docker, PostgreSQL, live import, LLM as compliance arbiter)
-

12. VALIDATION REQUIREMENTS (ALL MUST PASS)

Run these exact commands after the rebuild. All must pass before the work is considered complete.

Backend tests

```
powershell
```

```
$env:PYTHONPATH = 'backend'
```

```
backend\.venv\Scripts\python.exe -m pytest backend\tests -q
```

Required: All existing tests pass. New tests for the route refactoring must also pass. Target: ≥ 84 passed, 0 failed.

Frontend tests

```
powershell  
  
cd frontend  
npm test -- --run
```

Required: All 11 existing tests pass. Any new tests added also pass.

Frontend build

```
powershell  
  
cd frontend  
npm run build
```

Required: Zero errors. Output written to `frontend\dist\`.

Windows preflight

```
powershell  
  
scripts\preflight-windows.ps1 -AssumeYes
```

Required: All checks pass on a Windows 11 machine.

Smoke tests

```
powershell  
  
scripts\test-local-app-stack.ps1 -SkipDependencyInstall  
scripts\test-api-configuration-local.ps1 -SkipDependencyInstall
```

Required: All pass.

API health check

```
powershell  
  
Invoke-RestMethod http://127.0.0.1:8000/api/health  
Invoke-RestMethod http://127.0.0.1:8000/api/readiness  
Invoke-RestMethod http://127.0.0.1:8000/api/version
```

Required: All return 200. Version must show `1.2.0`.

Security scan

powershell

```
rg -n "sk-[A-Za-z0-9]|api[_-]?key|bearer |password=|secret=|token=|BEGIN PRIVAT  
-g "!frontend/package-lock.json" `  
-g "!backend/requirements-windows.txt" `  
-g "!.env.example"
```

Required: Zero real credential matches. Placeholder strings in `.env.example` and code comments are acceptable.

13. VERSION BUMP

Update version to `1.2.0` in:

- `VERSION` file (plain text)
- `VERSION.json` (json object with version, build, release_date, etc.)
- `backend/app/services/version.py`
- `CHANGELOG.md` (add a new entry for the clean rebuild)

Build identifier: `2026.06.14.1`

14. WORK ORDER (RECOMMENDED SEQUENCE)

Execute in this sequence to maintain a working state throughout:

1. **Set up clean project directory.** Copy all KEEP files. Verify no EXCLUDE files are present.
2. **Write** `backend/requirements-windows.txt`. Install into a fresh venv. Run backend tests.
3. **Refactor** `backend/app/api/routes.py` into domain routers. Run backend tests after each domain split. Do not proceed to the next domain until tests pass.
4. **Update** `backend/app/db/session.py` with SQLite WAL pragmas. Run backend tests.
5. **Update** `backend/app/main.py` router registration. Run backend tests.
6. **Extract TypeScript types** from `App.tsx` into `src/types/index.ts`. Run frontend tests.
7. **Extract API client** from `App.tsx` into `src/api/client.ts`. Run frontend tests.

8. **Extract views** one at a time from `App.tsx`. Run frontend tests after each extraction.
 9. **Extract layout components** (Sidebar, TopBar). Run frontend tests.
 10. **Create new components** (StatusBadge, Toast, ConfirmDialog, ErrorBanner, LoadingSpinner, timeliness components). Run frontend tests.
 11. **Implement UX improvements** (session persistence, upload progress, role-based default view, Windows 11 typography). Run frontend tests.
 12. **Update** `app.css` with Windows 11 font stack and status badge colors.
 13. **Write new** `AGENTS.md`.
 14. **Bump version to 1.2.0.**
 15. **Run full validation suite.** Fix any failures. Do not proceed until all pass.
 16. **Run** `scripts\build-windows-installer.ps1` to generate the release package.
-

15. WHAT SUCCESS LOOKS LIKE

A non-technical user named Marleigh (Clinical Director) opens Windows Explorer, double-clicks `Start-IZ-Clinical-Notes-Analyzer.cmd`, a browser window opens at `http://localhost:8000`, she logs in, and the first thing she sees is the Treatment Plan Timeliness work queue showing her 60+ active clients sorted by urgency (Overdue first, then Urgent, then Due Soon). She can:

1. Click any client row and see the evidence comparison panel (document due date vs. calculated due date vs. LOC-anchor date) without scrolling
2. Approve compliant clients or return non-compliant clients with a comment
3. Upload a new treatment plan PDF and watch a progress bar as it uploads
4. See a toast notification ("Upload successful") rather than a browser `alert()` dialog
5. Refresh the browser with F5 and NOT be logged out
6. Close the `.cmd` window and have port 8000 immediately available for the next launch
7. See all of the above in clean, professional Windows 11 typography (Segoe UI)

The app must run without any command-line interaction, Docker, PostgreSQL, or system-level configuration after initial Python setup.

End of Codex build goal. Begin with reading the full repo codebase before writing any code.